FPGA 電路設計與合成-Midterm

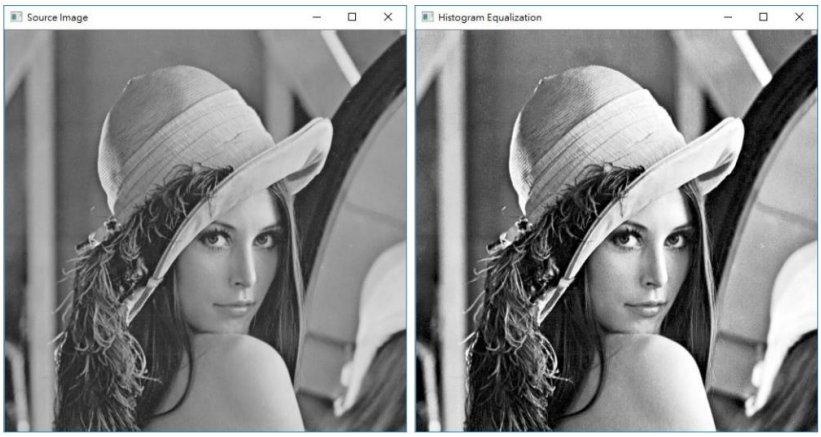
通訊四 411086040 曾義竣

一、YUV

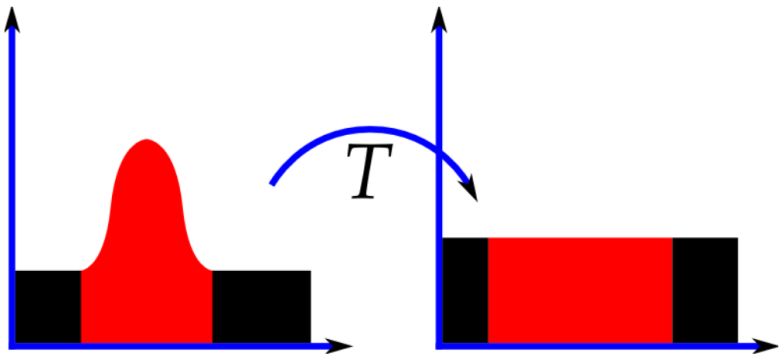
Y 通道代表影像的光亮程度（Luminance）。我們會將整個明亮程度限定在 0~255 之間，當資訊集中靠近 0 時，代表該圖呈現較暗成分，反之越接近 255 則代表整體較為明亮。而 U 和 V 表示色度（Chrominance），包含了影像的色彩資訊。U 和 V 分別是藍色和紅色的偏移量。色度信息相較於亮度更不敏感，所以可以壓縮或以更低解析度傳輸。

二、Histogram Equalization

Histogram，中文名稱為長方圖，在影像上，我們會應用於統計一張圖片裡面各數值出現的頻率，以 YUV 中的 Y 為舉例。Histogram Equalization 會統計灰階出現頻率後重新分配影像中的灰度值，使影像的灰度值分布接近均勻分布，從而增強影像對比度。從圖一可明顯看出經 Histogram Equalization 後的結果明顯對比增強許多。



圖一：左圖為原始影像，右圖為經 Histogram Equalization 的結果



圖二：原本灰階集中在中間對比不明顯，經 Histogram Equalization 後因數值同時往更暗語更亮移動使得整體對比增加。

在程式中，依據 Lab6 中的 yuv_filter.c 的邏輯，RGB 轉換成 YUV 後即可獲得 Y 分量的灰階資訊。因此，應在 rgb2yuv 步驟之後立即執行 Histogram Equalization，並將 rgb2yuv 的輸出指標作為 Histogram Equalization 的輸入，以增強影像的亮度對比效果。Histogram Equalization 的處理結果儲存在變數 yuv_eq 中，並作為 yuv_scale 的輸入，即可完成 Lab 6 加入 Histogram Equalization 的結果(圖四)。

```
int main () {
   // Dynamically allocate image buffers as they are rather large
   // and stack overflow may occur if statically allocated
   image_t *img_rgb = (image_t *)malloc(sizeof(image_t));
   image_t *img_restore = (image_t *)malloc(sizeof(image_t));

   // Read input image
   image_read(img_rgb);
   // Scale should be <=128 (128 corresponds to a scale of 1.0)
   yuv_filter(img_rgb, img_restore, 128, 128, 128);
   // Save output image
   image_write(img_rgb, img_restore);
}
```

圖三 : yuv_filter_test.c

```
   rgb2yuv   (in, yuv);
   histogram (yuv, yuv_eq);
   yuv_scale (     yuv_eq, scale, Y_scale, U_scale, V_scale);
   yuv2rgb   ( scale, out);
}
```

圖四 : yuv_filter 中主要函數(加入 Histogram Equalization)。

在 histogram 中，透過得到每個數值(tmp)對應該值次數加 1 來計算每個 Y 之灰階資訊(0~255)在影像出現頻率(圖五)，並將離散結果 histogram[i] 進行累加並除上 num_plxels 進行 normalize，形成亮度值的累積分佈函數 (CDF)。其機率定義為[0,1]。將累積的比例乘以 255 是為了將結果擴展到亮度值的範圍，即從 0 到 255。這樣可以將 0 到 1 的累積機率值擴展到新的灰階亮度值範圍，使每個亮度值對應一個 0 到 255 之間的值。最終將結果寫入 output 完成 Histogram Equalization。

```
    // Compute the histogram
HISTOGRAM_LOOP_X:
    for (i = 0; i < height; i++) {
      #pragma HLS loop_tripcount min=200 max=1920
HISTOGRAM_LOOP_Y:
        for (j = 0; j < width; j++) {
            #pragma HLS loop_tripcount min=200 max=1920
            count = in->channels.chY[i][j];
            histogram[count]++;
        }
    }
```

圖五 ： 計算每個 Y 之灰階資訊在影像出現頻率。

```
    // Calculate cumulative probability distribution and normalize it to 0-255
    float cumulative = 0.0;
HISTOGRAM_PROB_LOOP:
    for (i = 0; i < COLOR_LEN; i++) {
         #pragma HLS loop_tripcount min=200 max=1920
        cumulative += (float)histogram[i] / num_pixels;
        prob[i] = (int)(cumulative * 255);
    }
```

圖六 ： 將累積機率分佈變成可以直接用於像素值的轉換。

```
    // Map input image pixels to the new histogram-equalized values
HISTOGRAM_EQ_AND_MAP_LOOP_X:
    for (i = 0; i < height; i++) {
         #pragma HLS loop_tripcount min=200 max=1920
      HISTOGRAM_EQ_AND_MAP_LOOP_Y:
        for (j = 0; j < width; j++) {
            #pragma HLS loop_tripcount min=200 max=1920
            tmp = in->channels.chY[i][j];
            out->channels.chY[i][j] = prob[tmp];
            out->channels.chU[i][j] = in->channels.chU[i][j];
            out->channels.chV[i][j] = in->channels.chV[i][j];
        }
    }
```

圖七 ：YUV 結果 mapping 回 output。

三、Histogram Test (by .bmp)

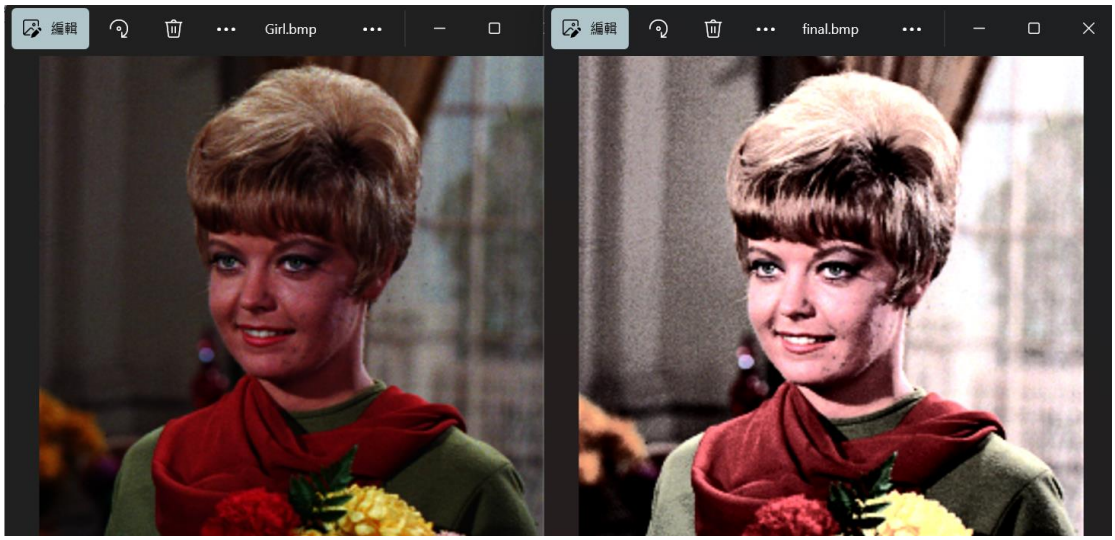
將測試檔 Girl.bmp 檔藉由 image_aux.c 中 fread 和 fwrite 的方式讀取原始 bmp 檔和寫入產生 final.bmp，其中因 bmp 中 RGB 順序為 B->G->R，因此除了須注意 bmp 的 header file 格式外，也需注意 RGB 寫入順序。

```
 for (i = 0; i<in_image->height; i++){
      for (j = 0; j<in_image->width; j++){//BMP B->G->R
      fread(&(in_image->channels.chB[i][j]), sizeof(unsigned char), 1, fp);
      fread(&(in_image->channels.chG[i][j]), sizeof(unsigned char), 1, fp);
      fread(&(in_image->channels.chR[i][j]), sizeof(unsigned char), 1, fp);
      }
  }
```

```
  for (i = 0; i < in_image->height; i++) {
      for (j = 0; j < in_image->width; j++) {
          fwrite(&out_image->channels.chB[i][j], sizeof(char), 1, fp);
          fwrite(&out_image->channels.chG[i][j], sizeof(char), 1, fp);
          fwrite(&out_image->channels.chR[i][j], sizeof(char), 1, fp);
      }
  }
```

圖八 ：fread 和 fwrite 為 B->G->R 順序讀取和寫入

最終將輸入 Girl.bmp 和輸出 final.bmp 進行比較，final.bmp 很明顯看出因使用 Histogram Equalization 的關係而產生較佳的對比效果。



圖九 ： 左邊為原始 Girl.bmp，右圖為經 Histogram Equalization 結果。

四、Pipeline

```
# Turning off the PIPELINE directive
set_directive_pipeline -off "yuv_scale/YUV_SCALE_LOOP_Y"
set_directive_pipeline -off "yuv2rgb/YUV2RGB_LOOP_Y"
set_directive_pipeline -off "rgb2yuv/RGB2YUV_LOOP_Y"
set_directive_pipeline -off "histogram/HISTOGRAM_LOOP_Y"
set_directive_pipeline -off "histogram/HISTOGRAM_EQ_AND_MAP_LOOP_Y"


set_directive_pipeline -off "yuv_scale/YUV_SCALE_LOOP_X"
set_directive_pipeline -off "yuv2rgb/YUV2RGB_LOOP_X"
set_directive_pipeline -off "rgb2yuv/RGB2YUV_LOOP_X"
set_directive_pipeline -off "histogram/HISTOGRAM_LOOP_X"
set_directive_pipeline -off "histogram/HISTOGRAM_EQ_AND_MAP_LOOP_X"

set_directive_pipeline -off "histogram/HISTOGRAM_PROB_LOOP"
```

圖十 : PIPELINE directive

Timing

Clock		solution2	solution1
ap_clk	Target	10.00 ns	10.00 ns
	Estimated	7.057 ns	7.057 ns

Latency

		solution2	solution1
Latency (cycles)	min	121083	447102
	max	7201104	28817903
Latency (absolute)	min	1.211 ms	4.471 ms
	max	72.011 ms	0.288 sec
Interval (cycles)	min	121084	447103
	max	7201105	28817904

Utilization Estimates

	solution2	solution1
BRAM_18K	1589	1587
DSP	20	13
FF	1975	1694
LUT	3827	3187
URAM	0	0

圖十一 : Compare with solution1(pipeline off) and solution2 (pipeline on)

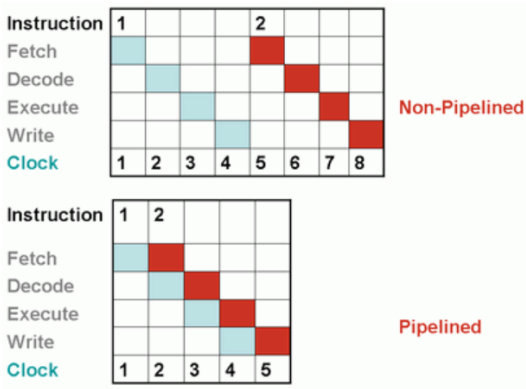
說明:

在圖十一中可以看出，當啟用 pipeline（solution2）時，不論是 min 還是 max，所產生的 Latency（cycles）、Latency（absolute）以及 Interval（cycles）均比關閉 pipeline（solution1）所花費的時間少，且兩者之間的差距與預設執行次數（loop_tripcount）呈正相關，執行次數越高，差距越明顯。這表明，啟用 pipeline 後並未經常產生閒置（idle）狀態。相反，由於使用 pipeline 需要並行處理數據，因此在 Utilization 中，DSP、FF 和 LUT 的使用量明顯高於 solution1。

Explain what is pipeline as detail as possible.

CPU 指令執行的過程可以分為幾個階段

- 取指 (Instruction Fetch): 從記憶體中提取指令
- 解碼 (Instruction Decode): 解碼指令，確定需要執行的操作
- 執行 (Execution): 執行指令所指定的操作
- 寫回 (Writeback): 將結果寫回到寄存器或記憶體中



圖十一 : Compare with pipeline and non-pipeline

Pipeline 的主要優勢是能夠提高計算機或處理單元的效能，在相同時間內，pipeline 會比 non-pipeline 做更多任務和數據，如此一來可以提升 CPU 的 Throughput，並行優勢也可避免某階段過度 idle 的問題。

但 Pipeline 可能也導致數據依賴 (Data Dependency)，假設使 pipeline 的情形下 :

指令 1： R2 = R1 + R3 (這條指令將寄存器 R1 和 R3 的值相加，結果存回 R2)
指令 2： R4 = R2 + R5 (這條指令將 R2 的結果與 R5 相加，結果存回 R4)

說明:

指令 2 依賴指令 1 的執行結果，也就是需要等指令 1 將 R2 的值寫回後，指令 2 才能進行執行。因此，如果指令 1 還未完成執行，指令 2 就不能進行計算，因此產生資料危障 (Data Hazard)。